

Tesi - Alessandro Porcheddu

Alessandro Ricci

June 2026

1 Introduzione e contesto applicativo

1.1 Dominio applicativo

Le centraline elettroniche, comunemente indicate con il termine Electronic Control Unit (ECU), rappresentano uno degli elementi fondamentali dei moderni sistemi automotive. Una ECU è un sistema embedded dedicato al controllo di specifiche funzioni del veicolo e, nel caso del motore, ha il compito di acquisire informazioni dai sensori, elaborarle secondo opportune strategie di controllo e generare i comandi necessari agli attuatori.

Nel corso degli anni, le ECU si sono evolute da semplici dispositivi di controllo a piattaforme software sempre più sofisticate. Nelle applicazioni automotive moderne e nelle competizioni motociclistiche di alto livello, le centraline non si limitano a gestire l'accensione o l'iniezione, ma integrano funzionalità avanzate quali telemetria, diagnostica, registrazione dei dati, configurazione dei parametri operativi e sistemi di protezione del motore. Questi strumenti consentono di osservare il comportamento del sistema, adattarne il funzionamento alle esigenze dell'applicazione e intervenire tempestivamente in presenza di condizioni anomale.

Tale evoluzione non ha però interessato in modo uniforme tutti i settori. Nel contesto dei motori monocilindrici a due tempi, molte delle soluzioni attualmente disponibili si basano ancora su tecnologie e architetture sviluppate diversi decenni fa. Le centraline impiegate in questo ambito offrono generalmente funzionalità limitate: la telemetria è spesso assente, gli strumenti diagnostici sono ridotti o inesistenti e le possibilità di configurazione risultano molto ristrette. In molti casi non è possibile modificare facilmente la curva di anticipo, adattare la risposta all'acceleratore o personalizzare il comportamento della centralina in funzione delle caratteristiche del motore e delle esigenze dell'utilizzatore.

Anche gli strumenti dedicati alla sicurezza e alla salvaguardia del propulsore risultano spesso limitati. L'assenza di meccanismi di monitoraggio e protezione rende più difficile individuare condizioni operative potenzialmente dannose e intervenire prima che possano causare guasti o degrado delle prestazioni. Di

conseguenza, molte attività di calibrazione e verifica continuano a dipendere da prove empiriche e dall'esperienza dell'operatore.

1.2 Idea di progetto

A partire dal contesto descritto, l'idea alla base del progetto è realizzare una ECU custom per motori monocilindrici a due tempi che non si limiti alla sola generazione dei segnali di controllo, ma che integri anche funzionalità normalmente associate a sistemi più evoluti. L'obiettivo è trasferire in questo ambito alcuni concetti ormai consolidati in altri settori automotive e motorsport, come la telemetria, la diagnostica, la configurazione dei parametri operativi e la protezione del motore.

La centralina viene quindi considerata come una piattaforma embedded completa, capace di acquisire informazioni dal motore, elaborarle in tempo reale e renderle disponibili per l'osservazione e la validazione del comportamento del sistema. Questa impostazione permette di superare il modello tradizionale della ECU come dispositivo chiuso e difficilmente ispezionabile, introducendo invece un'architettura più trasparente, configurabile ed estendibile.

Dal punto di vista funzionale, il progetto prevede che la ECU sia in grado di gestire i segnali provenienti dai sensori, determinare lo stato operativo del motore e generare i comandi necessari agli attuatori rispettando i vincoli temporali imposti dal funzionamento del propulsore. Parallelamente, il sistema deve poter raccogliere e trasmettere dati di funzionamento, rilevare condizioni anomale, conservare configurazioni persistenti e fornire strumenti utili alla verifica delle strategie implementate.

Un aspetto centrale dell'idea progettuale è la separazione tra controllo del motore e funzionalità di supporto. Il controllo real-time deve mantenere la priorità assoluta, mentre telemetria, diagnostica, configurazione e registrazione dei dati devono essere progettate in modo da non compromettere l'affidabilità della centralina. Questa distinzione consente di introdurre funzionalità avanzate senza trasformarle in un rischio per il corretto funzionamento del motore.

La ECU proposta non viene quindi intesa esclusivamente come un componente elettronico dedicato all'accensione o alla gestione di singoli attuatori, ma come il nucleo di un sistema più ampio. Attraverso l'integrazione con strumenti software esterni, essa può rendere il motore osservabile durante il funzionamento, permettere la modifica controllata dei parametri e supportare attività di analisi, calibrazione e validazione. In questo modo il progetto mira a costruire una base tecnica su cui sviluppare progressivamente funzionalità più avanzate, mantenendo una struttura coerente e adatta a un'applicazione embedded real-time.

1.3 Riferimenti architetturali e stato dell'arte

L'indipendenza tra logica applicativa e dettagli della piattaforma è un tema consolidato nel software automotive. AUTOSAR Classic, ad esempio, separa appli-

cazione, runtime e software di base; l'applicazione è in larga parte indipendente dall'hardware e accede ai servizi mediante interfacce definite [3]. L'architettura adotta una separazione calibrata sulle dimensioni del sistema: il codice che interpreta un sensore non conosce il driver ESP-IDF che ha prodotto il campione e i servizi applicativi dipendono da contratti stabili.

Un secondo riferimento è la validazione hardware-in-the-loop (HIL). In questo approccio il software esegue sul controllore reale mentre una sorgente esterna riproduce il comportamento dell'ambiente fisico. Abboush, Knieke e Rausch mostrano come HIL e iniezione di guasti possano sostenere la validazione real-time del software automotive [4]. Il simulatore del sistema genera il pick-up e gli ingressi analogici o digitali e li presenta alla ECU attraverso le normali interfacce elettriche. Le prove sono quindi ripetibili senza modificare la logica sottoposta a verifica.

Il terzo riferimento riguarda il digital twin. La letteratura sui motori a combustione interna associa il digital twin a monitoraggio, diagnosi e modelli predittivi alimentati dai dati del sistema fisico [5]. Lavori più recenti sui motori diesel combinano modelli fisici e tecniche di apprendimento per la stima dei parametri e il monitoraggio dello stato [6]. Nel sistema sviluppato il twin mantiene la rappresentazione persistente della ECU, delle revisioni installate e delle sessioni registrate. Su questa base esegue replay, confronto tra mappe e analisi diagnostica, mantenendo separati dati misurati, risultati ricalcolati e valori simulati.

1.4 Obiettivo e contributo della tesi

Il contributo è duplice. Sul piano progettuale viene definito un sistema completo, dai segnali fisici alla conservazione delle sessioni, con responsabilità separate. Sul piano sperimentale il sistema integra acquisizione e modellazione degli ingressi, controllo dell'accensione e degli attuatori, gestione delle mappe, pubblicazione di snapshot ed eventi, trasporto WebSocket, interfaccia web locale e servizi remoti per sessioni, aggiornamenti e digital twin.

La validazione segue l'intera catena: verifica che gli ingressi vengano acquisiti e qualificati, che le decisioni siano applicate con la temporizzazione richiesta, che la telemetria non blocchi il controllo e che sessioni e configurazioni rimangano tracciabili.

2 Analisi dei requisiti

2.1 Attori e confine del sistema

Il sistema principale è costituito dalla ECU, responsabile dell'acquisizione dei segnali, dell'elaborazione delle strategie di controllo e dell'attuazione dei comandi verso il motore.

Gli attori esterni principali sono quattro: il motore, il pilota, il tecnico e il

servizio remoto. Il motore rappresenta il processo fisico da controllare: da esso vengono acquisite le grandezze necessarie alla ricostruzione dello suo stato, come il regime di rotazione, la posizione angolare dell'albero motore, la posizione della farfalla, la temperatura dei gas di scarico e gli eventuali segnali associati alla detonazione. Allo stesso tempo, il motore è anche il destinatario finale delle azioni calcolate dalla ECU, come il comando di accensione della scintilla, la gestione del power jet e il controllo della valvola di scarico.

Il pilota interagisce con il sistema attraverso comandi diretti, legati alla guida del veicolo. Alcuni di questi segnali hanno carattere immediato e devono essere gestiti con priorità elevata, come la richiesta proveniente dal quick shifter o le variazioni rapide dell'apertura farfalla. Altri comandi, come la selezione della mappa o l'attivazione di strategie specifiche, non richiedono necessariamente la stessa latenza, ma devono comunque essere acquisiti e applicati in modo coerente con lo stato corrente della centralina.

Il tecnico rappresenta invece l'utente incaricato della configurazione, della calibrazione e della diagnosi del sistema. La sua interazione avviene tramite computer o smartphone collegandosi con un browser alla webapp fornita dalla ECU. Attraverso tale interfaccia il tecnico può osservare la telemetria, modificare parametri e mappe, verificare lo stato degli ingressi e degli attuatori, ed effettuare attività di test o manutenzione.

Il servizio remoto ha un ruolo distinto rispetto al ciclo di controllo. Esso può essere utilizzato per conservare dati, revisioni delle configurazioni, informazioni diagnostiche, statistiche sulla telemetria ed eventualmente per distribuire aggiornamenti software. La disponibilità della rete o del server non costituisce una condizione necessaria per il funzionamento della ECU.

2.2 Requisiti funzionali

- RF-001.** Il sistema deve acquisire un riferimento di posizione e velocità del motore per consentire il controllo sincronizzato delle funzioni motore.
- RF-002.** Il sistema deve determinare lo stato dinamico del motore, inclusi regime di rotazione, variazioni di velocità e stato di sincronizzazione.
- RF-003.** Il sistema deve validare la coerenza dei segnali utilizzati per il controllo motore e rilevare condizioni non plausibili.
- RF-004.** Il sistema deve consentire il funzionamento delle strategie motore solo quando le informazioni necessarie risultano affidabili.
- RF-005.** Il sistema deve calcolare il comando di accensione sulla base dello stato motore e delle strategie configurate.
- RF-006.** Il sistema deve supportare la configurazione dei parametri necessari alla sincronizzazione e al controllo del motore.

- RF-007.** Il sistema deve acquisire i principali parametri operativi del motore attraverso sensori dedicati.
- RF-008.** Il sistema deve normalizzare, validare e rendere disponibili i dati provenienti dai sensori alle funzioni di controllo.
- RF-009.** Il sistema deve supportare procedure di calibrazione dei sensori configurabili dall'utente.
- RF-010.** Il sistema deve applicare strategie di fallback in caso di indisponibilità o invalidità dei dati sensoriali.
- RF-011.** Il sistema deve gestire configurazioni persistenti e ripristinarle automaticamente all'avvio.
- RF-012.** Il sistema deve validare configurazioni e strategie prima della loro applicazione operativa.
- RF-013.** Il sistema deve utilizzare configurazioni di sicurezza predefinite in caso di dati mancanti, corrotti o non validi.
- RF-014.** Il sistema deve consentire la configurazione e il tuning delle strategie motore tramite interfacce dedicate.
- RF-015.** Il sistema deve fornire strumenti di monitoraggio in tempo reale dello stato del motore e della ECU.
- RF-016.** Il sistema deve rendere disponibili informazioni operative e diagnostiche tramite telemetria.
- RF-017.** Il sistema deve monitorare continuamente lo stato di salute dei sensori e delle funzioni interne.
- RF-018.** Il sistema deve rilevare, classificare e registrare condizioni di errore e anomalie operative.
- RF-019.** Il sistema deve acquisire parametri termici rilevanti per il monitoraggio dello stato del motore.
- RF-020.** Il sistema deve fornire informazioni sullo stato termico del motore e sulle relative tendenze evolutive.
- RF-021.** Il sistema deve rilevare fenomeni potenzialmente dannosi per il motore e renderli disponibili alle strategie di controllo.
- RF-022.** Il sistema deve supportare ingressi dedicati all'interazione del pilota con le strategie motore.
- RF-023.** Il sistema deve consentire la selezione e il cambio delle strategie operative durante il funzionamento.
- RF-024.** Il sistema deve supportare la gestione di più mappe o profili di funzionamento selezionabili.

- RF-025.** Il sistema deve fornire informazioni sullo stato corrente delle strategie attive e delle modalità operative.
- RF-026.** Il sistema deve supportare funzionalità di diagnostica e manutenzione del sistema.
- RF-027.** Il sistema deve consentire la simulazione delle principali condizioni operative per attività di verifica e validazione.
- RF-028.** Il sistema deve supportare la raccolta e la consultazione dei dati necessari all'analisi delle prestazioni e dei fault.
- RF-029.** Il sistema deve garantire la disponibilità delle informazioni necessarie al controllo, alla diagnostica e alla supervisione del motore.

2.3 Requisiti non funzionali

- RNF-001.** Le funzioni critiche di controllo motore devono operare con comportamento deterministico e prevedibile.
- RNF-002.** Le funzioni critiche devono essere isolate dalle funzionalità non essenziali quali interfacce utente, comunicazioni e servizi ausiliari.
- RNF-003.** Il sistema deve garantire la precisione temporale necessaria al corretto controllo del motore.
- RNF-004.** Il sistema deve minimizzare la latenza tra acquisizione degli eventi critici e relativa elaborazione.
- RNF-005.** Le attività non critiche non devono compromettere le prestazioni delle funzioni di controllo motore.
- RNF-006.** I meccanismi di comunicazione e telemetria devono essere non bloccanti rispetto alle funzioni critiche.
- RNF-007.** Il sistema deve avviarsi e operare in condizioni sicure fino alla verifica della propria integrità operativa.
- RNF-008.** Il sistema deve essere in grado di rilevare e recuperare da condizioni di blocco o malfunzionamento software.
- RNF-009.** Le configurazioni persistenti devono essere protette da corruzione e perdita di dati.
- RNF-010.** Le strategie di controllo devono mantenere comportamento coerente indipendentemente dal carico delle altre funzioni del sistema.
- RNF-011.** L'architettura software deve essere modulare e separare chiaramente responsabilità e domini funzionali.
- RNF-012.** Il sistema deve essere progettato per favorire manutenibilità, evoluzione e riutilizzo dei componenti software.

- RNF-013.** Le logiche di controllo devono essere verificabili e testabili indipendentemente dall'hardware finale.
- RNF-014.** Il sistema deve supportare attività di simulazione e validazione ripetibili e automatizzabili.
- RNF-015.** Le interfacce utente devono fornire una rappresentazione chiara dello stato operativo del sistema.
- RNF-016.** Ogni informazione resa disponibile dal sistema deve essere accompagnata da indicatori di validità e affidabilità quando applicabile.
- RNF-017.** Il sistema deve essere robusto rispetto a rumore, disturbi e condizioni operative non ideali.
- RNF-018.** La telemetria deve garantire coerenza, tracciabilità e contestualizzazione dei dati trasmessi.
- RNF-019.** In condizioni di risorse limitate, il sistema deve preservare prioritariamente le informazioni critiche per sicurezza e diagnostica.
- RNF-020.** Gli eventi e i dati registrati devono mantenere il corretto ordine temporale.
- RNF-021.** Le funzionalità diagnostiche avanzate devono essere configurabili e non interferire con il funzionamento operativo.
- RNF-022.** L'accesso alle funzioni critiche deve essere protetto da adeguati meccanismi di autorizzazione e conferma.
- RNF-023.** Il sistema deve adottare strategie di degradazione controllata in presenza di fault o perdita di informazioni.
- RNF-024.** L'architettura deve essere estendibile e consentire l'introduzione di nuove funzionalità senza modifiche sostanziali ai componenti esistenti.

2.4 Confine funzionale del sistema

Il sistema copre acquisizione, stima dello stato, controllo dell'accensione, attuazione ausiliaria, sicurezza, gestione delle mappe, telemetria, comandi, registrazione delle sessioni, digital twin e aggiornamento OTA. Le funzioni real-time risiedono sulla ECU; visualizzazione, conservazione e analisi sono distribuite tra client e server.

La piattaforma di prova comprende il condizionamento dei segnali e il simulatore necessari alla validazione HIL. La scheda dedicata, la qualificazione ambientale e l'integrazione meccanica sul veicolo appartengono al processo di industrializzazione e non modificano il confine funzionale dell'architettura.

3 Progettazione del sistema

3.1 Macro-architettura

L'architettura è composta da cinque sottosistemi. I primi quattro appartengono al sistema operativo; il quinto esiste per verificarlo.

Impianto fisico. Comprende motore, sensori, condizionamento dei segnali e attuatori. Produce grandezze continue ed eventi angolari e riceve comandi elettrici.

Nodo ECU. Acquisisce gli ingressi, stima lo stato del motore, applica controllo e sicurezza, comanda gli attuatori e pubblica telemetria. È l'unico sottosistema che deve rispettare le scadenze del motore.

Client locale. Visualizza stato ed eventi, invia richieste validate e conserva temporaneamente i dati necessari all'esperienza utente. Comunica con la ECU sulla rete locale.

Server e digital twin. Conserva identità, revisioni e sessioni. Offre confronto, replay e analisi storica. Non interviene nel controllo real-time.

Banco di simulazione. Riproduce segnali fisici controllabili e ripetibili. Dal punto di vista della ECU sostituisce uno o più sensori, non il software che li interpreta.

Il client svolge anche il ruolo di gateway verso il server. Questa scelta evita che la disponibilità di Internet diventi parte del ciclo di vita minimo della ECU e consente di mantenere disattivata la comunicazione remota durante il funzionamento critico. Le sessioni vengono accodate localmente e trasferite quando il client e il collegamento remoto sono disponibili.

3.2 Relazioni tra i sottosistemi

Il simulatore e i sensori reali comunicano con la ECU mediante segnali elettrici: livelli analogici, ingressi digitali, bus per convertitori e cattura degli eventi di pick-up. Questo collegamento appartiene al dominio fisico e attraversa gli stadi di protezione e condizionamento posti tra cablaggio e controllore.

ECU e client scambiano dati su un canale WebSocket locale. La direzione ECU–client trasporta snapshot, eventi, contatori e capacità dichiarate dal firmware; la direzione opposta trasporta richieste. Il protocollo bidirezionale non implica responsabilità simmetriche: il client chiede una modifica, mentre la ECU decide se sia valida e quando applicarla.

Il client inoltra al server sessioni e metadati attraverso una connessione non real-time. Il server restituisce storico, artefatti approvati e risultati di analisi. Per ogni sessione devono rimanere associati almeno firmware, configurazione, mappa e calibrazione. Senza questi riferimenti due curve simili potrebbero derivare da sistemi diversi e non sarebbero confrontabili.

3.3 Architettura interna della ECU

All'interno della ECU le dipendenze sono dirette dalla piattaforma verso il dominio, non nella direzione opposta. La decomposizione scelta comprende:

- **adattatori hardware**, che usano ADC, GPIO, SPI e periferiche di cattura e traducono i risultati in tipi indipendenti da ESP-IDF;
- **porte di acquisizione**, che definiscono le operazioni richieste dal dominio alle sorgenti fisiche o simulate;
- **oggetti e servizi di dominio**, che validano i campioni, calcolano stato di salute, debounce, timeout e grandezze derivate;
- **deposito sensoriale**, che pubblica istantanee coerenti (*snapshot*) e code limitate di eventi;
- **consumatori**, costituiti da controllo motore, sicurezza, telemetria, diagnostica e registrazione;
- **runtime**, che crea i task, assegna priorità e collega le implementazioni alle porte.

L'isolamento ha una conseguenza pratica: il sensore TPS non decide un limite di coppia e il quick shifter non porta direttamente a zero l'uscita CDI. Il primo pubblica una lettura o un fallback esplicito; il secondo pubblica una richiesta validata. La strategia di controllo e sicurezza conserva l'autorità sull'azione finale. In caso contrario, una modifica al debounce di un ingresso digitale potrebbe cambiare direttamente il comando dell'accensione.

3.4 Proprietà dei dati e concorrenza

Gli ingressi non hanno tutti la stessa semantica. Per TPS e temperature interessa il valore valido più recente; un aggiornamento non trasmesso può essere sostituito da uno più nuovo. Per cambio mappa, transizione di guasto e richiesta quick shifter conta invece l'evento: sostituirlo silenziosamente cambierebbe la storia osservata dal sistema. Il progetto usa quindi snapshot per lo stato e code limitate per gli eventi.

Ogni produttore scrive solo i dati che possiede. I consumatori leggono copie coerenti con contatori di sequenza o generazione. Un lettore lento può rilevare di avere saltato aggiornamenti, ma non mantiene il produttore in attesa. Questa regola è particolarmente rilevante quando telemetria e controllo vengono eseguiti su contesti o core differenti.

La telemetria è suddivisa per significato:

- il percorso di *stato* contiene l'ultima osservazione disponibile;
- il percorso di *eventi* conserva transizioni e richieste discrete;

- il percorso per *rivoluzione* consente di ricostruire il moto e le decisioni ciclo per ciclo;
- il percorso *diagnostico* contiene dati opzionali di sviluppo.

Quando la rete è congestionata, gli aggiornamenti ripetuti dello stato possono essere ridotti. Eventi di sicurezza e lacune nei dati registrati devono invece essere segnalati. Nessuna completezza del log giustifica il ritardo di un evento motore.

3.5 Controllo e sicurezza

Il sottosistema di controllo riceve uno snapshot degli ingressi e uno stato di sincronizzazione. Una macchina a stati distingue almeno inizializzazione, acquisizione del riferimento, funzionamento, taglio richiesto e condizione di allarme. Solo uno stato con riferimento angolare attendibile può produrre una schedulazione CDI.

La mappa fornisce un valore base; correzioni termiche, gestione della detonazione, limitatori e quick shifter producono richieste motivate. La strategia le combina e registra sia il risultato richiesto sia quello effettivamente comandato. Questa distinzione serve al digital twin: conoscere un anticipo finale senza sapere se sia stato limitato non consente di spiegare la decisione.

3.6 Digital twin come estensione del sistema

Il digital twin è collocato sul server e non sulla ECU. Per ogni dispositivo mantiene identità hardware, cronologia del firmware, configurazioni, mappe, calibrazioni, sessioni ed eventi. I dati vengono distinti in misurati, derivati, decisi, comandati, stimati e simulati.

Il twin ricostruisce le sessioni senza alterare i campioni originali, riesegue la logica di controllo sugli ingressi registrati e confronta l'uscita con mappe o parametri alternativi. Ogni risultato mantiene la provenienza e la revisione che lo ha prodotto. In questo modo il digital twin non coincide con un semplice grafico di telemetria, ma costituisce uno strumento di replay, confronto e diagnosi.

4 Il prototipo

4.1 Sottosistemi realizzati

Il prototipo integra tre catene funzionali. La prima acquisisce gli ingressi, stima lo stato del motore, calcola anticipo e riferimenti ausiliari e temporizza CDI, Power Jet e valvola di scarico. La seconda pubblica stato, eventi, decisioni e comandi verso l'interfaccia web. La terza registra le sessioni e le trasferisce al server, dove alimentano storico e digital twin.

sensori → stato motore → controllo → attuatori

stato e decisioni → telemetria → client → server

Le catene condividono tipi e identificativi, ma non le scadenze. Il percorso di controllo termina sulla ECU; telemetria, registrazione e analisi consumano copie coerenti e non possono ritardare la schedulazione degli output.

4.2 Piattaforma di prototipazione

La piattaforma è una scheda ESP32-S3 Mini basata su ESP32-S3FH4R2, con due core Xtensa LX7, 4 MB di flash, 2 MB di PSRAM, Wi-Fi a 2,4 GHz e logica a 3,3 V. La scheda rende disponibili ADC, GPIO, SPI, cattura temporizzata e connettività in un formato adatto al banco. I dati elettrici e le periferiche derivano dalla documentazione del produttore [7].

Il firmware è scritto in C++ sull'ambiente ESP-IDF e usa IDF FreeRTOS. La documentazione Espressif descrive l'implementazione SMP dual-core e la possibilità di assegnare affinità a uno specifico core [8]. Nel prototipo questa capacità permette di tenere acquisizione e pubblicazione sensoriale separate dal servizio HTTP/WebSocket. Priorità, sezioni critiche e carico sono verificati nelle prove temporali, mentre le periferiche hardware eseguono cattura e generazione degli eventi con intervento minimo della CPU.

Il banco usa circuiti di condizionamento per adattare i segnali ai livelli a 3,3 V della scheda. Alimentazione, protezioni e interfacce verso sensori e attuatori restano esterne al modulo di calcolo, così da mantenere separata la valutazione del firmware da quella elettrica.

4.3 Ingressi e harness

Ogni famiglia di ingressi è esposta al dominio attraverso una porta. La sorgente può essere reale oppure sintetica; i servizi a valle non cambiano. Il profilo di test usa sorgenti sintetiche deterministiche per TPS, temperature, pick-up, quick shifter, selettore mappa e knock. Lo stesso percorso di validazione, pubblicazione e controllo viene esercitato con ingressi reali o simulati.

Gli ingressi fisici sono:

- TPS su un canale ADC, con conversione calibrata in millivolt;
- quick shifter su un ingresso digitale attivo basso;
- selettore di mappa su un ingresso digitale con debounce e validazione dello stato;
- pick-up già condizionato a onda quadra, acquisito mediante una periferica di cattura sul fronte di discesa;
- EGT tramite convertitore per termocoppia su bus SPI;

- temperatura acqua tramite partitore NTC e canale ADC;
- knock tramite interfaccia dedicata e finestra sincronizzata con la rivoluzione del motore.

4.4 Modello sensoriale

Una lettura contiene valore, timestamp monotono, validità, qualità, salute, sequenza e bit di guasto applicabili. Il deposito sensoriale produce uno snapshot coerente degli ingressi e mantiene code limitate per gli eventi. Il TPS applica calibrazione e controlli di intervallo; il pick-up rifiuta intervalli impossibili e perde la sincronizzazione quando gli eventi diventano obsoleti; gli ingressi digitali implementano debounce e riarmo.

In presenza di un TPS non valido il servizio pubblica un fallback esplicito al 70 per cento. La strategia di sicurezza riconosce l'origine sostitutiva del valore, applica i limiti previsti dal profilo operativo e registra la condizione in telemetria.

Le temperature pubblicano stato e richieste di protezione, ma non eseguono lo spegnimento. Analogamente, il sottosistema knock pubblica misure finestrate e qualità; la strategia di accensione decide se e come ritardare l'anticipo. Questa separazione conserva una sola autorità sul comando finale.

4.5 Controllo motore e attuatori

Gli eventi di pick-up aggiornano regime, sincronizzazione e identificativo di rivoluzione. La macchina a stati abilita il controllo soltanto quando il riferimento è attendibile. La mappa attiva fornisce l'anticipo base; correzioni termiche, knock, limitatore e quick shifter producono il comando finale e un codice che ne descrive la motivazione.

L'angolo viene convertito in un intervallo temporale rispetto al pick-up e programmato su una periferica hardware. Power Jet e valvola di scarico usano uscite PWM calcolate dalle rispettive mappe. Il sottosistema di attuazione pubblica separatamente riferimento richiesto, comando applicato, saturazioni e inibizioni, così che telemetria e digital twin possano ricostruire la decisione.

4.6 Telemetria e interfaccia web

Il componente di telemetria raccoglie snapshot ed eventi senza leggere direttamente i driver. Il contratto espone stato dei sensori, sincronizzazione, salute, contatori, riepilogo knock, eventi ordinati, decisioni di controllo e comandi degli attuatori. Le informazioni destinate alla visualizzazione vengono raggruppate in piccoli batch e serializzate in JSON.

JSON è più oneroso di un formato binario, ma rende il contratto leggibile e semplifica l'ispezione con strumenti comuni. Per la dashboard il compromesso è adeguato. I record per rivoluzione e le sessioni vengono invece codificati in forma binaria compatta prima del trasferimento al server.

Il server HTTP/WebSocket gira sulla ECU. Pubblica un messaggio di capacità, invia la telemetria quando il trasporto è pronto e non preleva nuovi batch durante disconnessione o backpressure. Lo stesso server distribuisce gli asset statici compressi dell'interfaccia. La Web UI adatta il contratto ai propri modelli di visualizzazione, conserva uno storico locale limitato e mostra letture, salute ed eventi recenti. I componenti dell'interfaccia non interpretano direttamente il JSON ricevuto: il parser costituisce il confine tra protocollo e presentazione. Dalla stessa interfaccia l'utente seleziona e modifica le mappe, consulta le sessioni, avvia i comandi diagnostici e gestisce gli aggiornamenti firmware. Le richieste vengono validate dalla ECU prima dell'applicazione.

4.7 Mappe, sessioni e aggiornamenti

Le mappe di accensione e Power Jet sono memorizzate in memoria non volatile e usano punti RPM con interpolazione lineare. Il sistema verifica ordinamento, intervalli e completezza prima di salvare una revisione. L'attivazione sostituisce l'intera configurazione in un confine sicuro, evitando che il controllo osservi una mappa modificata solo in parte.

La registrazione distingue la telemetria live dai dati di sessione. La prima è ottimizzata per l'aggiornamento della dashboard; i secondi conservano record per rivoluzione, eventi, decisioni e comandi in forma binaria. Il client inoltra i blocchi al server insieme a identità ECU, firmware, mappa, configurazione e calibrazione. Il server verifica completezza e sequenza prima di aggiornare il digital twin.

L'aggiornamento OTA confronta la versione installata con l'artefatto approvato, verifica integrità e autenticità dell'immagine e scrive la partizione inattiva. Il nuovo firmware viene confermato dopo l'avvio corretto; in caso contrario il bootloader ripristina la versione precedente.

4.8 Scelte implementative rilevanti

Quattro scelte caratterizzano l'implementazione.

La prima è l'uso di tempo iniettabile e sorgenti deterministiche. Timeout, obsolescenza e debounce possono essere provati senza attendere il tempo reale. La seconda è il limite esplicito di code e storico: un sovraccarico produce un contatore o una perdita riconoscibile. La terza è l'assenza di dipendenze da rete e storage nel dominio sensoriale. La quarta è la dichiarazione delle capacità del protocollo, che permette a client e firmware di evolvere senza assumere che ogni versione esponga gli stessi dati.

Le sorgenti vengono selezionate mediante profili validati, così che una configurazione non possa associare un driver a un ingresso elettrico incompatibile. JSON è riservato alla telemetria live, mentre la registrazione usa un formato compatto e versionato. Le operazioni che modificano mappe o firmware

richiedono autenticazione e autorizzazione; gli aggiornamenti sono verificati prima dell'avvio e prevedono il ripristino della versione precedente.

5 Validazione

5.1 Obiettivo della validazione

La validazione deve rispondere a una domanda più stretta di “il software funziona”: quali requisiti sono sostenuti da un'evidenza ripetibile e a quale livello? Sono considerati quattro livelli. I test sul calcolatore esercitano il dominio senza hardware; la configurazione interamente sintetica esegue i servizi sul dispositivo con ingressi deterministici; la modalità mista verifica i singoli collegamenti fisici; il banco HIL misura il comportamento integrato di ingressi, controllo, attuatori e telemetria.

5.2 Test su calcolatore

I test del dominio sensoriale eseguiti sul calcolatore coprono i casi seguenti:

- conversione dei timestamp di cattura e gestione del ritorno del contatore;
- calibrazione TPS, intervalli invalidi, obsolescenza, fallback e recupero;
- debounce, riarmo e condizioni iniziali di quick shifter e map switch;
- stima RPM, acquisizione e perdita della sincronizzazione, intervalli impossibili e rapido aumento di regime;
- guasti termici, richieste di protezione e recupero senza comando diretto degli attuatori;
- misure knock mancanti, sature o arretrate;
- coerenza degli snapshot, sequenze e comportamento delle code piene;
- instradamento delle sorgenti sintetiche e reali attraverso l'harness;
- interpolazione delle mappe, composizione delle correzioni e transizioni della macchina a stati;
- validazione, persistenza e attivazione atomica delle mappe;
- inibizione dell'accensione con riferimento angolare non valido.

I test della telemetria verificano la composizione dello stato più recente, l'ordinamento degli eventi e l'esposizione dei contatori di overflow. I test del server verificano serializzazione, dichiarazione delle capacità, comportamento in disconnessione e backpressure, risoluzione degli asset statici e rifiuto dei percorsi non validi. Sul client vengono verificati parser, adattamento al modello di visualizzazione, storico limitato, conservazione dell'ordine degli eventi e validazione dei comandi.

I test dei servizi remoti coprono registrazione delle sessioni, associazione delle revisioni, immutabilità dei campioni, replay e aggiornamento OTA con ripristino. L'esecuzione automatica associa i risultati a revisione del codice, compilatore e configurazione usati nella prova.

5.3 Prove sul dispositivo

In modalità interamente sintetica il firmware esegue sulla ESP32-S3 e pubblica periodicamente uno snapshot in formato CSV o attraverso la telemetria. La sequenza attesa è specifica: il regime diventa valido dopo un numero sufficiente di catture, TPS e temperature cambiano secondo gli stimoli, e gli eventi di quick shifter e cambio mappa compaiono in punti ripetibili. Questa prova collega servizi, deposito e runtime senza introdurre l'incertezza del cablaggio.

La modalità mista serve a verificare un confine elettrico alla volta. Una rampa tra 0 e 3,3 V sull'ingresso analogico deve coprire l'intervallo calibrato del TPS; l'ingresso del quick shifter deve produrre una sola richiesta validata dopo debounce e riarmo; il selettore deve pubblicare il cambio della richiesta fisica; un'onda quadra condizionata sull'ingresso di cattura produce il regime coerente con la frequenza e diventa obsoleta quando gli impulsi cessano. Le prove termiche verificano lettura nominale, termocoppia aperta, guasto del convertitore e superamento delle soglie; la catena knock viene esercitata con finestre valide, assenti e sature.

Il banco HIL applica profili di accelerazione, impulsi mancanti, disturbi, corto verso massa o alimentazione e guasti dei convertitori. La strumentazione esterna misura la differenza tra il riferimento angolare simulato e l'uscita CDI e verifica duty cycle e frequenza degli attuatori ausiliari. Le stesse prove sono ripetute con telemetria e trasferimento delle sessioni attivi, così da misurare l'isolamento del percorso real-time.

5.4 Tracciabilità rispetto ai requisiti

La Tabella 1 collega ogni requisito funzionale all'evidenza di validazione corrispondente. I test sul calcolatore verificano la logica; le prove sul dispositivo e HIL verificano i confini elettrici e temporali; le prove end-to-end coprono client e server.

Table 1: Copertura dei requisiti funzionali.

Requisito	Evidenza di validazione	Stato
RF-001	Cattura del pick-up, stima RPM e acquisizione della sincronizzazione	Soddisfatto
RF-002	Stima RPM, perdita e recupero della sincronizzazione, rapido aumento di regime	Soddisfatto
RF-003	Controlli di intervallo, intervalli impossibili del pick-up e rifiuto dei valori non plausibili	Soddisfatto

Requisito	Evidenza di validazione	Stato
RF-004	Abilitazione del controllo solo con riferimento attendibile e inibizione con riferimento angolare non valido	Soddisfatto
RF-005	Interpolazione della mappa, composizione delle correzioni e transizioni della macchina a stati	Soddisfatto
RF-006	Profili validati delle sorgenti, validazione e persistenza dei parametri	Soddisfatto
RF-007	Test dei sensori con sorgenti sintetiche e collegamenti fisici (modalità mista)	Soddisfatto
RF-008	Calibrazione, controlli di intervallo e coerenza degli snapshot	Soddisfatto
RF-009	Calibrazione TPS e validazione dei comandi di configurazione	Soddisfatto
RF-010	Fallback TPS esplicito, obsolescenza e recupero	Soddisfatto
RF-011	Persistenza e attivazione delle mappe, ripristino delle revisioni all'avvio	Soddisfatto
RF-012	Validazione, persistenza e attivazione atomica delle mappe	Soddisfatto
RF-013	Valori di sicurezza predefiniti, fallback espliciti e code piene	Soddisfatto
RF-014	Validazione, applicazione e risposta ai comandi di configurazione	Soddisfatto
RF-015	Viste, stato di salute, eventi e storico locale	Soddisfatto
RF-016	WebSocket, sequenze, congestione e rilevamento delle lacune	Soddisfatto
RF-017	Stato di salute dei sensori e contatori diagnostici	Soddisfatto
RF-018	Obsolescenza, guasti, eventi di transizione, fallback e recupero	Soddisfatto
RF-019	Prove termiche: lettura nominale, termocoppia aperta, guasto del convertitore e soglie	Soddisfatto
RF-020	Pubblicazione dello stato termico e delle richieste di protezione	Soddisfatto
RF-021	Misure knock mancanti, sature o arretrate e finestra sincronizzata	Soddisfatto
RF-022	Quick shifter e selettore di mappa: debounce, riarmo e richieste validate	Soddisfatto
RF-023	Cambio mappa e selezione della strategia in punti ripetibili	Soddisfatto
RF-024	Persistenza, validazione e attivazione atomica delle mappe	Soddisfatto
RF-025	Decisioni di controllo e codici di motivazione in telemetria	Soddisfatto
RF-026	Comandi diagnostici, contatori e aggiornamento OTA con ripristino	Soddisfatto
RF-027	Sorgenti sintetiche deterministiche e banco HIL con fault injection	Soddisfatto

Requisito	Evidenza di validazione	Stato
RF-028	Registrazione delle sessioni, immutabilità dei campioni e replay	Soddisfatto
RF-029	Coerenza degli snapshot, telemetria e ricostruzione delle sessioni	Soddisfatto

5.5 Valutazione dei requisiti non funzionali

RNF-013 e RNF-014 sono verificati compilando il dominio senza ESP-IDF e controllando il tempo mediante sorgenti deterministiche, così da rendere le logiche di controllo testabili indipendentemente dall'hardware e le prove ripetibili. RNF-019 è verificato saturando code e storici: in condizioni di risorse limitate il sistema preserva le informazioni critiche ed espone contatori e lacune senza crescita non limitata della memoria. RNF-010 è verificato attraverso le regole di singolo scrittore e le istantanee coerenti, che mantengono il comportamento del controllo indipendente dal carico delle altre funzioni. La disconnessione di client e server conferma RNF-002 e RNF-006, poiché l'isolamento delle funzioni critiche e la telemetria non bloccante lasciano invariata la strategia di controllo attiva.

Per RNF-001, RNF-003 e RNF-004 le misure HIL confrontano latenza e jitter con il budget temporale del controllo, sia a rete inattiva sia durante il massimo carico di telemetria. Le prove di concorrenza confermano RNF-005, poiché acquisizione, schedulazione e pubblicazione sensoriale non attendono i consumatori. La coerenza degli snapshot verifica inoltre che ogni lettura conservi validità e salute (RNF-016). RNF-018 e RNF-020 sono verificati ricostruendo una sessione dalla relativa identità ECU, versione firmware, mappa, configurazione e calibrazione, mantenendo ordine e provenienza dei dati. Le prove HIL con disturbi, corto verso massa o alimentazione e guasti dei convertitori sostengono RNF-017, mentre il fallback esplicito degli ingressi conferma la degradazione controllata richiesta da RNF-023. L'aggiornamento OTA con verifica dell'artefatto, autorizzazione e ripristino della versione precedente sostiene RNF-007, RNF-009 e RNF-022.

I risultati mostrano che le funzioni del sistema sono coperte da prove coerenti con il loro livello: logica sul calcolatore, integrazione sul dispositivo, tempi e segnali sul banco HIL, persistenza nelle prove end-to-end.

6 Conclusioni e sviluppi futuri

6.1 Risultati

Il progetto definisce una ECU per motori monocilindrici come parte di un sistema più ampio: sensori e attuatori sul veicolo, client locale, servizio remoto e banco di simulazione. La separazione tra questi nodi stabilisce che il controllo rimane sulla ECU, mentre interfaccia e server osservano, configurano e conservano i dati senza entrare nel percorso real-time.

Il prototipo realizza l'intera catena descritta. Sorgenti reali o simulate attraversano gli stessi servizi e producono snapshot ed eventi qualificati; il controllo calcola e comanda accensione e attuatori; telemetria e Web UI rendono osservabili misure e decisioni; il server conserva sessioni e revisioni e fornisce replay e confronto mediante il digital twin. La validazione collega queste funzioni ai requisiti e verifica l'isolamento del percorso real-time.

6.2 Valutazione critica delle scelte

La scheda ESP32-S3 offre un buon rapporto tra capacità di calcolo, periferiche e connettività. Il front-end esterno mantiene condizionamento e protezioni fuori dal modulo di calcolo; una scheda dedicata può in seguito integrare questi elementi e ridurre cablaggio e ingombro. Il Wi-Fi rimane estraneo alle funzioni essenziali e può essere disattivato senza interrompere il controllo.

Il fallback TPS fisso rende esplicita la condizione degradata e mantiene deterministico il comportamento. Una politica dipendente dal punto operativo può offrire limiti più conservativi, ma aumenta il numero di casi da calibrare e verificare. Per questo il valore sostitutivo resta configurabile e viene sempre accompagnato dallo stato di qualità.

Il contratto JSON mantiene leggibile la telemetria live, mentre il formato binario versionato contiene il costo delle registrazioni per rivoluzione. La doppia rappresentazione richiede un dizionario semantico comune, ma evita di forzare lo stesso formato su visualizzazione e archiviazione. Il client come gateway protegge l'autonomia della ECU; l'accodamento locale gestisce i periodi nei quali il gateway non è disponibile.

Il digital twin mantiene separate osservazione, replay e simulazione. Questa distinzione impedisce che un risultato ricalcolato venga interpretato come misura fisica e rende confrontabili configurazioni diverse. I modelli predittivi possono usare la stessa base dati, ma conservano una validazione distinta dal controllo eseguito sulla ECU.

6.3 Lavori futuri

Gli sviluppi individuati estendono prestazioni, integrazione e capacità di analisi senza modificare i confini principali del sistema:

- mappe a due ingressi RPM-TPS con strumenti di confronto e ottimizzazione automatica delle superfici;
- strategie knock adattive che distinguano fondo meccanico, transitori e detonazione ripetuta;
- ampliamento del banco HIL con profili derivati da sessioni reali e campagne automatiche di fault injection;
- analisi temporale estesa a diverse condizioni di carico e temperatura;

- scheda dedicata con protezioni, condizionamento, alimentazione e connettori adatti all'ambiente del veicolo;
- integrazione con reti CAN e strumenti diagnostici automotive;
- modelli termici e predittivi aggiuntivi nel digital twin;
- campagne di calibrazione su motori con cilindrata e configurazione differenti.

Queste estensioni riusano gli stessi contratti tra acquisizione, controllo, telemetria e analisi, mantenendo verificabili le nuove funzioni in modo indipendente.

References

- [1] Link Engine Management, *Why an Aftermarket ECU? Selecting an Aftermarket Engine Management System*. <https://linkecu.com/new-to-ecus/why-an-aftermarket-ecu/>.
- [2] International Energy Agency, *Vehicle software and software-defined vehicles*, 20 maggio 2026. <https://www.iea.org/reports/vehicle-software-and-software-defined-vehicles>.
- [3] AUTOSAR, *Classic Platform*. <https://www.autosar.org/standards/classic-platform>.
- [4] M. Abboush, C. Knieke e A. Rausch, *A Virtual Testing Framework for Real-Time Validation of Automotive Software Systems Based on Hardware in the Loop and Fault Injection*, *Sensors*, vol. 24, n. 12, 2024, doi: 10.3390/s24123733.
- [5] V. D. Tran, P. Sharma e L. H. Nguyen, *Digital Twins for Internal Combustion Engines: A Brief Review*, *Journal of Emerging Science and Engineering*, vol. 1, n. 1, pp. 29–35, 2023, doi: 10.61435/jese.2023.5.
- [6] K. Nath, V. Kumar, D. J. Smith e G. E. Karniadakis, *A Digital Twin for Diesel Engines: Operator-infused Physics-Informed Neural Networks with Transfer Learning for Engine Health Monitoring*, *Engineering Applications of Artificial Intelligence*, 2026, doi: 10.1016/j.engappai.2026.114052.
- [7] Espressif Systems, *ESP32-S3 Series Datasheet*. https://documentation.espressif.com/esp32-s3_datasheet_en.pdf.
- [8] Espressif Systems, *FreeRTOS (IDF) – ESP-IDF Programming Guide*. https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/freertos_idf.html.